



Guide

An Adobe® Photoshop® CSS processor.

Version 0.9.4 beta

(experimental version)

2013-12-15

Index

1	Introduction.....	5
2	Installing and Starting the Script	6
3	Use Cases.....	7
3.1	Named Properties	7
3.2	Photoshop Dialog Windows	7
3.3	Calculations and Expressions.....	8
3.4	Color Map and Size Map Images	9
3.5	Layouts via CSS Templates	10
3.6	Automatic Sprite Generation	11
3.7	PreLoad Images	13
3.8	Plugins and Automatic Image Generation	14
4	Main Dialog Window	16
5	Basic Ideas of μCSS	16
5.1	The -μcss-Property	16
5.2	The -μcss Rules	17
5.2.1	::-μcss-init Rule	17
5.2.2	::-μcss-start Rule.....	17
5.2.3	::-μcss-end Rule	17
5.2.4	::-μcss-exit Rule	17
5.3	Simple CSS Compiling	17
5.4	Template CSS Compiling.....	17
5.5	JavaScript Statements	17
6	μ Object	18
6.1	μ.\$ Object	18
6.2	μ.options Object.....	18
6.2.1	μ.options.sprites Object	18
6.2.1.1	μ.options.sprites.creation	18
6.2.1.2	μ.options.sprites.relPath	18
6.2.1.3	μ.options.sprites.fileName	18
6.2.1.4	μ.options.sprites.format.....	18
6.2.1.5	μ.options.sprites.compression	18
6.2.1.6	μ.options.sprites.interlaced	18
6.2.1.7	μ.options.sprites.quality.....	18
6.2.1.8	μ.options.sprites.padding.....	18
6.2.1.9	μ.options.sprites.log Object	18
6.2.2	μ.options.createPreLoadRule	18
6.2.3	μ.options.colorOutputByName	19
6.3	μ.runs Object	19
6.4	μ.cache Object.....	19
6.4.1	μ.cache.data	19
6.4.1.1	μ.cache.data.plugins.....	19
6.5	μ Methods	19
6.5.1	System Methods.....	19
6.5.1.1	μ.Log	19
6.5.1.2	ShowURL.....	19
6.5.1.3	μ.SetPreference	19
6.5.1.4	μ.GetPreference	19
6.5.2	Rule Methods	19
6.5.3	Block Methods.....	19

6.5.4	Property Methods	19
6.5.4.1	μ.SetProperty	19
6.5.4.2	μ.SetColor	20
6.5.5	Color Methods.....	20
6.5.5.1	μ.Alpha	20
6.5.5.2	μ.Red.....	20
6.5.5.3	μ.Green.....	20
6.5.5.4	μ.Blue.....	20
6.5.5.5	μ.Hue	20
6.5.5.6	μ.Brightness.....	20
6.5.5.7	μ.Saturation	21
6.5.5.8	μ.Lighten.....	21
6.5.5.9	μ.Darken	21
6.5.5.10	μ.MultiplyColors	21
6.5.5.11	μ.NegMultiplyColors.....	21
6.5.5.12	μ.SetColorsOfImagePoints.....	21
6.5.6	Sprites.....	21
6.5.6.1	μ.Sprite	21
6.5.7	Preloading Images	22
6.5.7.1	μ.PreLoadImage.....	22
6.5.8	Cursors.....	22
6.5.8.1	μ.DefCursor	22
6.5.8.2	μ.Cursor	23
6.5.9	File System Operation Methods.....	23
6.5.9.1	μ.OpenFile	23
6.5.9.2	μ.CopyFile	23
6.5.9.3	μ.DeleteFile	23
6.5.9.4	μ.CopyFolder	23
6.5.9.5	μ.DeleteFolder.....	23
6.5.10	FTP Methods.....	23
6.5.10.1	μ.SyncFTPFolder	23
6.5.10.2	μ.SyncLocalFolder.....	24
7	μμ Object.....	26
7.1	μμ.\$ Object.....	26
7.2	μμ.topCSSFiles.....	26
8	Working with Colors.....	27
8.1	Defining Colors	27
8.2	Working with Alpha Channels	28
8.3	Color Calculations.....	28
9	PlugIns and Extensions	29
9.1	ButtonCreator PlugIn.....	29
9.1.1	Create	29
10	Future Expansions	31
11	Release History	32
12	Legal Information	33
12.1	MIT Licence Terms.....	33
12.2	Trademarks.....	34
12.3	Included Libraries	34
12.3.1	json.jsxinc	34
12.3.2	Bitpack.jsxinc	34
13	Donate	35

1 Introduction

μCSS is a Photoshop tool for CSS file processing and Web graphics generation. By including special manufacturer properties to CSS files, which includes JavaScript statements, a maximum of flexibility is given. These are the main benefits of μCSS:

- Always browser conform CSS files.
- Named CSS properties.
- Calculation of CSS properties by JavaScript expressions.
- File generation via templates.
- SpriteMaps support.
- Support of preloading CSS images.
- Integrated plugin and extension system.
- Improves the interface between programmers and graphic designers.
- Integrated FTP client incl. folder sync functions (e.g. to transfer local generated CSS files to servers).
- Integrated cache system to speed up compiling/generation speed.

2 Installing and Starting the Script

To install μCSS just extract the package and put it anywhere on your hard drive. Then create an alias (Mac) or a link (Windows) of the file μCSS/μCSS.jsx and put it to your script folder of Photoshop.

After this and a restart of Photoshop the script appears at the script menu of Photoshop.

Another way to execute the μCSS script is to use *Adobe ExtendScript Toolkit*. After extraction of the package you can open and start the μCSS.jsx file directly in this tool.

When the μCSS script is started, the [main μCSS dialog window](#) is shown, where you can take the actions

3 Use Cases

In the project folder you will find some examples. You can test it by yourself. To do this, you have to compile the CSS files with the Photoshop μCSS script. Start the script and select the corresponding CSS file.

3.1 Named Properties

The simplest way to use μCSS is to use it to define property variables, so a CSS property is named and only defined once in a CSS file.

Before compiling the file named `_css_properties.css`:

```
::-μcss-init{
  -μcss:μ.$.textcolor="#ff0000";
  -μcss:μ.$.backcolor="#0000ff";
}

div.mydiv{
  padding:5px;
  font-size:24px;
  font-weight:600;
  width:300px;
  height:100px;
  -μcss:μ.SetColor(μ.$.textcolor);
  -μcss:μ.SetBorderColor(μ.$.backcolor);
  -μcss:μ.SetBackgroundColor(μ.$.backcolor);
  color:#0000ff;
  background-color:#00ff00;
  border:10px #000000 solid;
  border-radius:10px;
}
```

After compiling the file named `_css-properties.css`:

```
/* This CSS is compiled by μCSS Photoshop Tool. */
::-μcss-init{
  -μcss:μ.$.textcolor="#ff0000";
  -μcss:μ.$.backcolor="#0000ff";
}

div.mydiv{
  padding:5px;
  font-size:24px;
  font-weight:600;
  width:300px;
  height:100px;
  -μcss:μ.SetColor(μ.$.textcolor);
  -μcss:μ.SetBorderColor(μ.$.backcolor);
  -μcss:μ.SetBackgroundColor(μ.$.backcolor);
  color: #ff0000;
  background-color: #0000ff;
  border:10px #0000ff solid;
  border-radius:10px;
}
```

3.2 Photoshop Dialog Windows

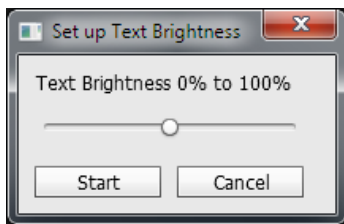
It is possible to use all kinds of Photoshop-JavaScript methods. In this way you can also use Photoshop Dialog Windows:

```
::-μcss-init{
  -μcss:μ.$.dlg=new Window(
    "dialog«text:'Set up Text Brightness','"
    "bounds:[100,100,300,200],"+
    "butok:Button«bounds:[10,70,90,90],"+
    "text:'Start',properties:«name:'ok'»»,"+
    "butcancel:Button«bounds:[100,70,180,90],"+
    "text:'Cancel',properties:«name:'cancel'»»,"+
    "thetitle:StaticText«bounds:[10,10,180,30],"+
    "text:'Text Brightness 0% to 100%'»»,"+
  )
}
```

```
"TextBrightness:Slider«minvalue:0,maxvalue:100,"+
"value:50,bounds:[10,30,180,50]»»"+
">";
-μcss:if(μ.$.dlg.show()!=1)μ.AbortCompiling();
}
div.mydiv{
font-size:24px;
font-weight:600;
width:300px;
height:60px;
-μcss:μ.SetColor(μ.HSL(180,100,μ.$.dlg.TextBrightness.value));
color:#fef8f0;
background-color:black;
padding:20px;
}
```

Note: Special characters like “{};” have to substitute in statements to prevent CSS syntax conflicts.

On running this file with μCSS, a Photoshop dialog window will be shown, where the user can set up the parameters with which the new CSS file will be created:



For more information about the features of Adobe ScriptUI have a look at [ScriptUI for dummies](#).

3.3 Calculations and Expressions

Because the μCSS statements based on JavaScript any types of calculation and logical operation are possible. μCSS includes also many methods to support color and size calculations.

File examples/calculations.css before compiling:

```
::-μcss-init{
-μcss:μ.$.mycolor="#ff0000";
-μcss:μ.$.mypadding=100;
-μcss:μ.$.textshadow=false;
}
::-μcss-exit{
-μcss:μ.ShowURL("calculations.html");
}
div.mydiv{
font-size:24px;
font-weight:600;
width:300px;
height:60px;
-μcss:μ.SetColor(μ.Lighten(μ.$.mycolor,"#808080"));
-μcss:μ.SetBackgroundColor(μ.$.mycolor);
color:#000000;
background-color:#0000ff;
-μcss:μ.SetPadding(Math.floor(μ.$.mypadding*0.2)+"px");
padding:10px;
-μcss:μ.$.textshadow?μ.SetTextShadow("-1px -1px 2px rgba(0,0,0,1)":μ.RemoveProperty("text-shadow");
text-shadow:-1px -1px 2px rgba(0,0,0,1);
}
```

File examples/calculations.css after compiling:

```
/* This CSS is compiled by μCSS Photoshop Tool. */
::-μcss-init{
-μcss:μ.$.mycolor="#ff0000";
-μcss:μ.$.mypadding=100;
-μcss:μ.$.textshadow=true;
}
::-μcss-exit{
-μcss:μ.ShowURL("calculations.html");
}
div.mydiv{
font-size:24px;
font-weight:600;
```

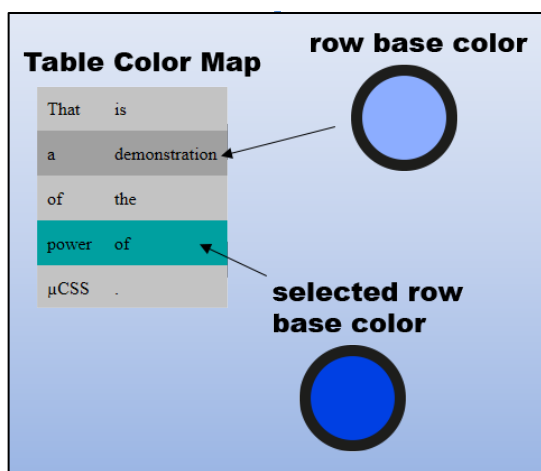


```
width:300px;
height:60px;
-μcss:μ.SetColor(μ.Lighten(μ.$.mycolor,"#808080"));
-μcss:μ.SetBackgroundColor(μ.$.mycolor);
color:#ff7f7f;
background-color:#ff0000;
-μcss:μ.SetPadding(Math.floor(μ$.mypadding*0.2)+"px");
padding:20px;
-μcss:μ$.textshadow?μ.SetTextShadow("-1px -1px 2px rgba(0,0,0,1)":μ.RemoveProperty("text-shadow");
}
```

3.4 Color Map and Size Map Images

Because μCSS is based on Photoshop all features of Photoshop will be available for μCSS, including the access to any type of images. There a special methods inside μCSS which supports this features. Have a look to this example:

This image is given where the colors for the new CSS-file are defined:



You can change the colors at the layers included in group “layout colormaps” (the black bordered circles) to change the colors for the corresponding layout.

The source μCSS-script file looks like this:

```
::-μcss-init{
-μcss:μ.SetColorsOfImagePoints("color_map_example.psd",[[["μ$.tablebasecolor","skin color-
maps/skin_graycyan",352,93],[["μ$.tablehovercolor","skin colormaps/skin graycyan",313,343]]]);
}
table.mytable{
border-collapse:collapse;
}
table.mytable>tbody>tr:nth-child(even){
-μcss:μ.SetBackgroundColor(μ$.tablebasecolor);
background-color:#000000;
}
table.mytable>tbody>tr:nth-child(odd){
-μcss:μ.SetBackgroundColor(μ.Lighten(μ$.tablebasecolor,"#a0a0a0"));
background-color:#000000;
}
table.mytable>tbody>tr:nth-child(even):hover{
-μcss:μ.SetBackgroundColor(μ$.tablehovercolor);
background-color:#000000;
}
table.mytable>tbody>tr:nth-child(odd):hover{
-μcss:μ.SetBackgroundColor(μ.Lighten(μ$.tablehovercolor,"#a0a0a0"));
background-color:#000000;
}
table.mytable>tbody>tr>td{
padding:10px;
}
```

After running μCSS with the μCSS script above, the result should be like this:

```
/* This CSS is compiled by μCSS Photoshop Tool. */
::-μcss-init{
  -μcss:μ.SetColorsOfImagePoints("color_map_example.psd",[[["μ.$tablebasecolor","skin color-
maps/skin_graycyan",352,93],[["μ.$tablehovercolor","skin colormaps/skin_graycyan",313,343]]]);
}
table.mytable{
  border-collapse:collapse;
}
table.mytable>tbody>tr:nth-child(even){
  -μcss:μ.SetBackgroundColor(μ.$tablebasecolor);
  background-color:#9999ff;
}
table.mytable>tbody>tr:nth-child(odd){
  -μcss:μ.SetBackgroundColor(μ.Lighten(μ.$tablebasecolor,"#a0a0a0"));
  background-color:#bfbfff;
}
table.mytable>tbody>tr:nth-child(even):hover{
  -μcss:μ.SetBackgroundColor(μ.$tablehovercolor);
  background-color:#0033cc;
}
table.mytable>tbody>tr:nth-child(odd):hover{
  -μcss:μ.SetBackgroundColor(μ.Lighten(μ.$tablehovercolor,"#a0a0a0"));
  background-color:#5f7fdf;
}
table.mytable>tbody>tr>td{
  padding:10px;
}
```

Note: Not only colors can read from images, also sizes and booleans.

3.5 Layouts via CSS Templates

The simplest way to compile a CSS file is to use the same file as input and output. Another interest usage is to use CSS-template files to generate new CSS files for various layouts. Therefore it is possible to read in a CSS file and save the compiled CSS file into another file.

First we need the template css file, e.g. `css_by_template.css`.

To define different layouts, we need each a css file to define the parameters of the layouts, e.g. `μ.css_by_template_layout_A.css` and `μ.css_by_template_layout_B.css`. This files contains only one `-μcss-init` rule to define the parameters and the template css file to use:

File `μ.css_by_template_layout_A.css`:

```
::-μcss-init{
  /* Dependent Files */
  -μcss:μ.DependentCSSFile("css_by_template.css","css_by_template_layout_A.css");
  /* Layout A */
  -μcss:μ.$textcolor="#ff0000";
  -μcss:μ.$backcolor="#0000ff";
}
```

File `μ.css_by_template_layout_B.css`:

```
::-μcss-init{
  /* Dependent Files */
  -μcss:μ.DependentCSSFile("css_by_template.css","css_by_template_layout_B.css");
  /* Layout B */
  -μcss:μ.$textcolor="#ff00ff";
  -μcss:μ.$backcolor="#00ff00";
}
```

Both file above have to compile with μCSS. After this two new files are generated, one for layout A and one for layout B:

File `css_by_template_layout_A.css`:

```
/* This CSS is compiled by μCSS Photoshop Tool. */
div.mydiv{
  padding:5px;
  font-size:20px;
```

```
font-weight:600;
width:500px;
height:100px;
-μcss:μ.SetColor(μ.$.textcolor);
-μcss:μ.SetBorderColor(μ.$.backcolor);
-μcss:μ.SetBackgroundColor(μ.$.backcolor);
color:#ff0000;
background-color:#0000ff;
border:10px #0000ff solid;
border-radius:10px;
}
```

File css_by_template_layout_B.css:

```
/* This CSS is compiled by μCSS Photoshop Tool. */
div.mydiv{
padding:5px;
font-size:20px;
font-weight:600;
width:500px;
height:100px;
-μcss:μ.SetColor(μ.$.textcolor);
-μcss:μ.SetBorderColor(μ.$.backcolor);
-μcss:μ.SetBackgroundColor(μ.$.backcolor);
color:#ff00ff;
background-color:#00ff00;
border:10px #00ff00 solid;
border-radius:10px;
}
```

3.6 Automatic Sprite Generation

One of the highlights of μCSS is the feature to generate sprite images from a CSS file.

If this CSS file will be compiled...

```
::-μcss-init{
-μcss:μ.options.sprites.creation=true;
-μcss:μ.options.sprites.relPath="./imgs";
}
div.loginbutton{
display:inline-block;
-μcss:μ.Sprite("imgs/aqua/but_login_normal.png");
background-image:url(imgs/aqua/but_login_normal.png);
width:55px;;
height:55px;;
}
div.loginbutton:hover{
display:inline-block;
-μcss:μ.Sprite("imgs/aqua/but_login_hover.png");
background-image:url(imgs/aqua/but_login_hover.png);
width:55px;;
height:55px;;
}
div.helpbutton{
display:inline-block;
-μcss:μ.Sprite("imgs/aqua/but_help_normal.png");
background-image:url(imgs/aqua/but_help_normal.png);
width:55px;;
height:55px;;
}
div.helpbutton:hover{
display:inline-block;
-μcss:μ.Sprite("imgs/aqua/but_help_hover.png");
background-image:url(imgs/aqua/but_help_hover.png);
width:55px;;
height:55px;;
}
div.settingsbutton{
display:inline-block;
-μcss:μ.Sprite("imgs/aqua/but_settings_normal.png");
background-image:url(imgs/aqua/but_settings_normal.png);
width:55px;;
height:55px;;
}
div.settingsbutton:hover{
display:inline-block;
-μcss:μ.Sprite("imgs/aqua/but_settings_hover.png");
background-image:url(imgs/aqua/but_settings_hover.png);
width:55px;;
height:55px;;
}
```

...Photoshop will generate this image (and also a @2x-Retina version)...



...and change the CSS properties, so that the new generated sprite image will be used instead of the former single images:

```
/* This CSS is compiled by µCSS Photoshop Tool. */
::-µcss-init{
  -µcss:µ.options.sprites.creation=true;
  -µcss:µ.options.sprites.relPath="./imgs";
}
div.loginbutton{
  display:inline-block;
  -µcss:µ.Sprite("imgs/aqua/but_login_normal.png");
  background-image:url(imgs/sprites.png);
  background-image:image-set(url(imgs/sprites.png)1x,url(imgs/sprites@2x.png)2x);
  background-repeat:no-repeat;
  background-position:-0px -0px;
  width:55px;
  height:55px;
}
div.loginbutton:hover{
  display:inline-block;
  -µcss:µ.Sprite("imgs/aqua/but_login_hover.png");
  background-image:url(imgs/sprites.png);
  background-image:image-set(url(imgs/sprites.png)1x,url(imgs/sprites@2x.png)2x);
  background-repeat:no-repeat;
  background-position:-55px -0px;
  width:55px;
  height:55px;
}
div.helpbutton{
  display:inline-block;
  -µcss:µ.Sprite("imgs/aqua/but_help_normal.png");
  background-image:url(imgs/sprites.png);
  background-image:image-set(url(imgs/sprites.png)1x,url(imgs/sprites@2x.png)2x);
  background-repeat:no-repeat;
  background-position:-0px -55px;
  width:55px;
  height:55px;
}
div.helpbutton:hover{
  display:inline-block;
  -µcss:µ.Sprite("imgs/aqua/but_help_hover.png");
  background-image:url(imgs/sprites.png);
  background-image:image-set(url(imgs/sprites.png)1x,url(imgs/sprites@2x.png)2x);
  background-repeat:no-repeat;
  background-position:-55px -55px;
  width:55px;
  height:55px;
}
div.settingsbutton{
  display:inline-block;
  -µcss:µ.Sprite("imgs/aqua/but_settings_normal.png");
  background-image:url(imgs/sprites.png);
  background-image:image-set(url(imgs/sprites.png)1x,url(imgs/sprites@2x.png)2x);
  background-repeat:no-repeat;
  background-position:-110px -0px;
  width:55px;
  height:55px;
}
div.settingsbutton:hover{
  display:inline-block;
  -µcss:µ.Sprite("imgs/aqua/but_settings_hover.png");
  background-image:url(imgs/sprites.png);
  background-image:image-set(url(imgs/sprites.png)1x,url(imgs/sprites@2x.png)2x);
  background-repeat:no-repeat;
  background-position:-110px -55px;
  width:55px;
  height:55px;
}
```

3.7 PreLoad Images

A technique to speed up the user interactions on web pages is to preloading all important images. μCSS can collect all marked images in to a single CSS rule, so the images are loaded immediately when the html file is loaded in to the browser:

```
/* This CSS is compiled by μCSS Photoshop Tool. */
::-μcss-init{
  -μcss:μ.options.createPreLoadRule=true;
}
div.loginbutton{
  display:inline-block;
  -μcss:μ.PreLoadImage("imgs/aqua/but_login_normal.png");
  background-image:url("imgs/aqua/but_login_normal.png");
  width:55px;
  height:55px;
}
div.loginbutton:hover{
  display:inline-block;
  -μcss:μ.PreLoadImage("imgs/aqua/but_login_hover.png");
  background-image:url("imgs/aqua/but_login_hover.png");
  width:55px;
  height:55px;
}
div.helpbutton{
  display:inline-block;
  -μcss:μ.PreLoadImage("imgs/aqua/but_help_normal.png");
  background-image:url("imgs/aqua/but_help_normal.png");
  width:55px;
  height:55px;
}
div.helpbutton:hover{
  display:inline-block;
  -μcss:μ.PreLoadImage("imgs/aqua/but_help_hover.png");
  background-image:url("imgs/aqua/but_help_hover.png");
  width:55px;
  height:55px;
}
div.settingsbutton{
  display:inline-block;
  -μcss:μ.PreLoadImage("imgs/aqua/but_settings_normal.png");
  background-image:url(imgs/sprites.png);
  width:55px;
  height:55px;
}
div.settingsbutton:hover{
  display:inline-block;
  -μcss:μ.PreLoadImage("imgs/aqua/but_settings_hover.png");
  background-image:url("imgs/aqua/but_settings_hover.png");
  width:55px;
  height:55px;
}
-μcss-preload{
  background-image:url(imgs/aqua/but_login_normal.png) -99999px -99999px;
  background-image:image-
  set(url(imgs/aqua/but_login_normal.png)1x,url(imgs/aqua/but_login_normal@2x.png)2x);
  background-image:url(imgs/aqua/but_login_hover.png) -99999px -99999px;
  background-image:image-
  set(url(imgs/aqua/but_login_hover.png)1x,url(imgs/aqua/but_login_hover@2x.png)2x);
  background-image:url(imgs/aqua/but_help_normal.png) -99999px -99999px;
  background-image:image-
  set(url(imgs/aqua/but_help_normal.png)1x,url(imgs/aqua/but_help_normal@2x.png)2x);
  background-image:url(imgs/aqua/but_help_hover.png) -99999px -99999px;
  background-image:image-
  set(url(imgs/aqua/but_help_hover.png)1x,url(imgs/aqua/but_help_hover@2x.png)2x);
  background-image:url(imgs/aqua/but_settings_normal.png) -99999px -99999px;
  background-image:image-
  set(url(imgs/aqua/but_settings_normal.png)1x,url(imgs/aqua/but_settings_normal@2x.png)2x);
  background-image:url(imgs/aqua/but_settings_hover.png) -99999px -99999px;
  background-image:image-
  set(url(imgs/aqua/but_settings_hover.png)1x,url(imgs/aqua/but_settings_hover@2x.png)2x);
  display:none;
}
```

The rule -μcss-pre load is generated by μCSS. To make the image loading active, simply the html file has to uses this class:

```
<html>
<head>
  <title>μCSS Images PreLoad Example</title>
  <link rel="stylesheet" media="screen" href="image_preload.css">
</head>
```

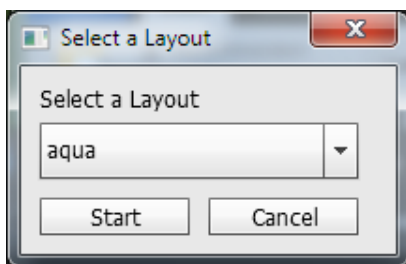
```
<body>
  <div class="-µcss-preload"></div>
  That's a demonstration of the power of µCSS.
  <br/>
  <div class="loginbutton"></div>
  <div class="helpbutton"></div>
  <div class="settingsbutton"></div>
</body>
</html>
```

3.8 PlugIns and Automatic Image Generation

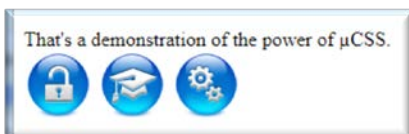
µCSS supports plugins. One of the standard plugins is ButtonCreator. With this plugin it is possible to create buttons with same icons in different layouts, controlled by a CSS file and µCSS:

```
::-µcss-init{
-µcss:µ.$.dlg=new Window("dialog«text:'Select a Layout', bounds:[100,100,300,200], "+
    "butok:Button«bounds:[10,70,90,90], text:'Start', "+
    "properties:«name:'ok'»», "+
    "butcancel:Button«bounds:[100,70,180,90], text:'Cancel', "+
    "properties:«name:'cancel'»», "+
    "thetitle:StaticText«bounds:[10,10,180,30], text:'Select a Layout', "+
    "csslayout:DropDownList«bounds:[10,30,180,60], "+
    "properties:«items:['alu','aqua'], selection:text:'alu'»»"+
    "»i");
-µcss:µ.$.dlg.csslayout.selection=1;
-µcss:if(µ.$.dlg.show()!=1)µ.AbortCompiling();
-µcss:µ.$.layout=µ.$.dlg.csslayout.selection.text;
-µcss:µ.plugins.ButtonCreator.Create("drafts/buttons.psd",µ.$.layout,"imgs/"+µ.$.layout,true);
}
div.loginbutton{
display:inline-block;
-µcss:µ.SetBackgroundImage("imgs/"+µ.$.layout+"/but_login_normal@2x.png");
width:55px;
height:55px;
background-image:url(imgs/alu/but_login_normal.png);
background-image:image-
set(url(imgs/alu/but_login_normal.png)1x,url(imgs/alu/but_login_normal@2x.png)2x);
}
div.loginbutton:hover{
display:inline-block;
-µcss:µ.SetBackgroundImage("imgs/"+µ.$.layout+"/but_login_hover@2x.png");
width:55px;
height:55px;
background-image:url(imgs/alu/but_login_hover.png);
background-image:image-
set(url(imgs/alu/but_login_hover.png)1x,url(imgs/alu/but_login_hover@2x.png)2x);
}
...
}
```

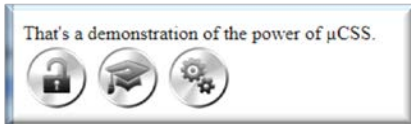
After compiling start with this CSS file via µCSS, a dialog appears to choice the new layout of the CSS file:



µCSS will then generate all button images for the selected layout:

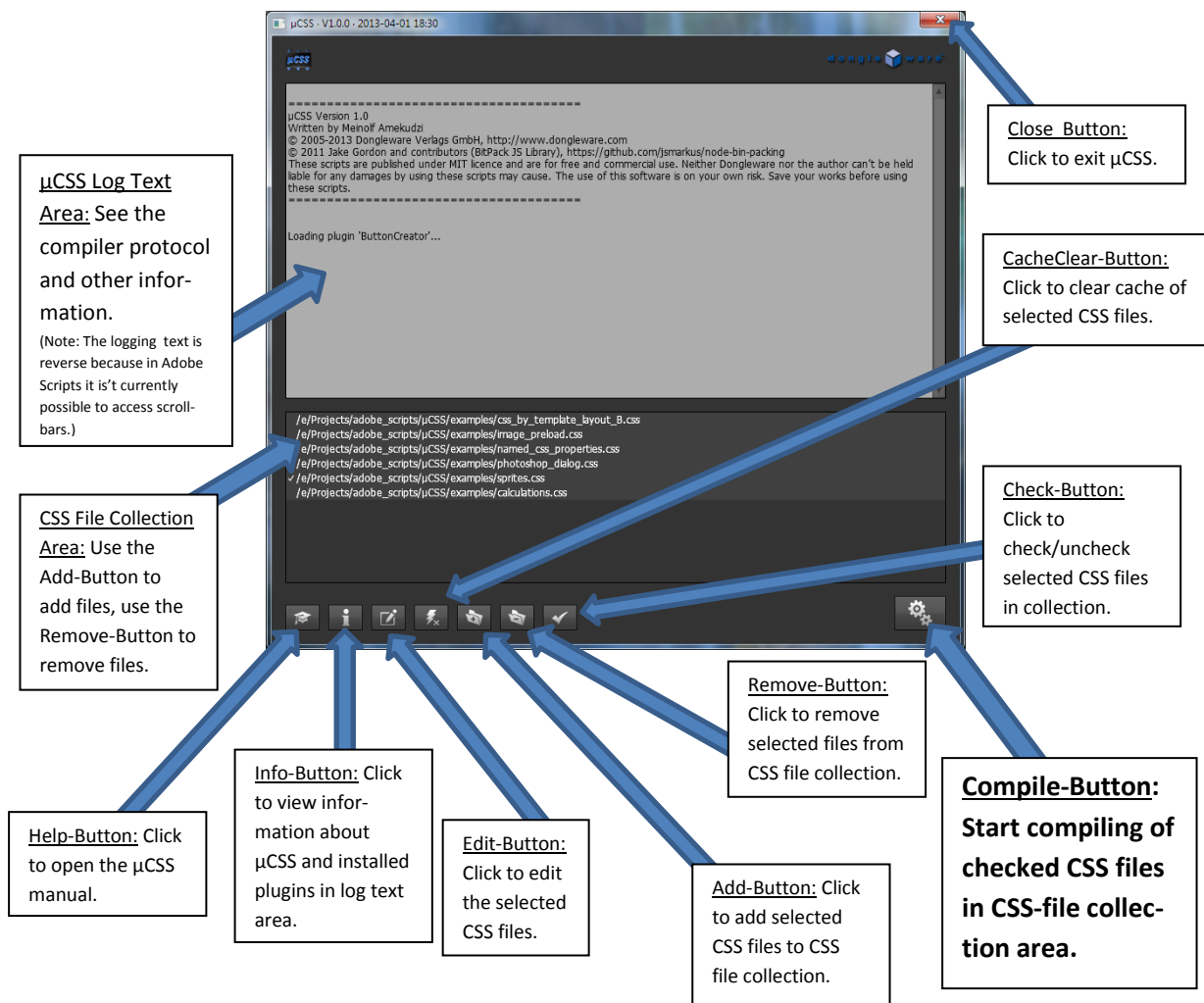


or



4 Main Dialog Window

After calling the script, the μCSS main dialog window is shown:



This window is modal, so you have to close it to use Photoshop in normal way. All μCSS actions are taken using the buttons on this window. Before you can compile a CSS file, you have to add it to the CSS File Collection Area by pressing the Add-Button. If you click the Compile-Button all checked CSS files will be compiled. CSS files can be checked and unchecked by selecting the files and pressing the Check-Button.

To use the Edit-Button, you have to configure your system, so that CSS-files (suffix .css) are opened in a text editor by default (e.g. Windows context menu `open with...`).

5 Basic Ideas of μCSS

5.1 The -μcss-Property

The basic idea of μCSS is to define JavaScript Code to CSS files to

Of course the JavaScript is not executed while the HTML page is rendered in a browser. The code is executed by the developer of the HTML page.

```
-μcss:[JavaScript Statement];
```


5.2 The -μcss Rules

There are some special rules using by μCSS, the `::-μcss-init`, `::-μcss-start`, `::-μcss-end` and the `::μcss-exit`-rule.

5.2.1 `::-μcss-init` Rule

The μCSS-properties of this rule will be executed when the CSS file is loaded. If there are multiple CSS files, which e.g. are loaded via the method `μ.DependentCSSFile`, all initialization rules of all files will be executed before any μCSS-property of any other of the rules in the CSS files are executed.

5.2.2 `::-μcss-start` Rule

The μCSS-properties of this rule will be executed before the CSS file start with the compilation of the the enclosed CSS rules.

5.2.3 `::-μcss-end` Rule

The μCSS-properties of this rule will be executed after the file is compiled and before it saved to disk.

5.2.4 `::μcss-exit` Rule

The μCSS-properties of this rule will be executed after the compiled file is saved to disk. So this rule is recommend to make FTP synchronizations or other operations based on the saved CSS file.

5.3 Simple CSS Compiling

A simple μCSS compiling is given by a single file which includes normally an initialization rule `::-μcss-init` and will be saved with the same name as the source CSS file.

5.4 Template CSS Compiling

To run a template compiling to create new layouts based on a given CSS file, you have to use the *DependentCSSFile* method and an extra file which only includes an initial μCSS rule. At the initialization μCSS rule (`::-μcss-init`) all dependent CSS files and the destination files will be defined.

The filename of the initial CSS file should be named like `μ.mycsslayoutname.css` (beginning with `μ.`).

5.5 JavaScript Statements

To ensure the conformation of CSS syntax so that all browsers can interpret the μCSS files without any problem, some characters can't use inside a μCSS property. To circumvent this problem it is necessary to replace these characters with none conflict characters:

JavaScript Character	Replacement μCSS Character
{	«
}	»
;	i

Most often it is not necessary to use these characters because there is only one JavaScript statement per line and it is closed by the semicolon of the CSS property.

6 μ Object

This is the base global object of μCSS and is created by μCSS before a CSS file will be compiled. It is valid for the compilation of one CSS file.

6.1 μ.\$ Object

The μ.\$ property is reserved for user properties, valid for the compilation of the current CSS file.

6.2 μ.options Object

This object contains the μCSS compiler options, valid for the compilation of the current CSS file.

6.2.1 μ.options.sprites Object

6.2.1.1 μ.options.sprites.creation

This boolean value defines whether sprite images should be generate. Default value is false. See also chapter [Sprite Methods](#) for more details.

6.2.1.2 μ.options.sprites.relPath

The relative path where the generated sprite images should be saved. This path is relative to the current CSS output file. Change this string to change the output path.

6.2.1.3 μ.options.sprites.fileName

....

6.2.1.4 μ.options.sprites.format

....

6.2.1.5 μ.options.sprites.compression

....

6.2.1.6 μ.options.sprites.interlaced

....

6.2.1.7 μ.options.sprites.quality

....

6.2.1.8 μ.options.sprites.padding

....

6.2.1.9 μ.options.sprites.log Object

....

6.2.1.9.1 μ.options.sprites.log.relPath

....

6.2.1.9.2 μ.options.sprites.log.filename

....

6.2.2 μ.options.createPreLoadRule

This boolean value defines whether the -μcss-preload rule should be generate. Default value is false. See also chapter [Preloading Images](#) for more details.

6.2.3 **μ.options.colorOutputByName**

This boolean vale defines whether color names („red“, „green“ etc.) instead of color values (like #808080 or rgb(0,0,0)) should be used on setting of color properties.

6.3 **μ.runs Object**

This objects contains runtime information of the current compiling CSS file.

6.4 **μ.cache Object**

A cache system is included in μCSS to speed up compiling. E.g. image sizes and modification times are cached to prevent a repeatedly loading of images by sprite creation and sprite parameters are cached to prevent not needed sprite generations in general. The cache is stored in the temporary system folder of the current user. The cache of the selected files can be cleared with the corresponding button on the main dialog window of μCSS.

6.4.1 **μ.cache.data**

....

6.4.1.1 **μ.cache.data.plugins**

....

6.5 **μ Methods**

6.5.1 **System Methods**

6.5.1.1 **μ.Log**

6.5.1.2 **ShowURL**

6.5.1.3 **μ.SetPreference**

6.5.1.4 **μ.GetPreference**

6.5.2 **Rule Methods**

6.5.3 **Block Methods**

6.5.4 **Property Methods**

6.5.4.1 **μ.SetProperty**

This is the universal method to set or change a property in a block.

`μ.SetProperty ( name, value )`

If the property with the same name already exists, the value of the existing property will be changed.
If the property does not exist, a new property will be added.

Parameter	Description	Default Value
name	The name of the property. This will automatically set to lower case characters.	./.
value	The value of the property, normal a string.	./.

6.5.4.2 μ.SetColor

Sets the color property in current block.

```
μ.SetColor ( [color] )
```

Parameter	Description	Default Value
color	The color to set, any string or integer (Working with Colors)	#00000000

6.5.5 Color Methods

These methods give you same basic color calculation functions. See chapter [Working with Colors](#) to get information how to work with this methods.

6.5.5.1 μ.Alpha

Text text text.

```
μ. Alpha ( alphaValue )
```

6.5.5.2 μ.Red

Text text text.

```
μ.Red ( color )
```

6.5.5.3 μ.Green

Returns the green channel of a color.

```
μ.Green ( color, [format] )
```

Parameter	Description	Default Value
color	A color according to the color μCSS color format.	./.
format	The output format of the green channel value.	CHANNEL_HEX

6.5.5.4 μ.Blue

Text text text.

```
μ.Blue ( color )
```

6.5.5.5 μ.Hue

Text text text.

```
μ. Hue ( color )
```

6.5.5.6 μ.Brightness

Text text text.

```
μ.Brightness ( color )
```

6.5.5.7 μ.Saturation

Text text text.

```
μ.Saturation ( color )
```

6.5.5.8 μ.Lighten

Text text text.

```
μ.Lighten ( color, percent )
```

6.5.5.9 μ.Darken

Text text text.

```
μ.Darken ( color, percent )
```

6.5.5.10 μ.MultiplyColors

Text text text.

```
μ.MultiplyColors ( colorA, colorB )
```

6.5.5.11 μ.NegMultiplyColors

Text text text.

```
μ.NegMultiplyColors (colorA, colorB )
```

6.5.5.12 μ.SetColorsOfImagePoints

6.5.6 Sprites

μCSS includes Web Sprite methods to generate sprite images based on given CSS files. A sprite image is a big image including all small images used in classes of your CSS file. The benefit is to have only one image to load. The properties of background-position, background-repeat, image-width and -height will be automatically set by μCSS.

6.5.6.1 μ.Sprite

Define a background image as sprite.

```
μ.Sprite ( url, [offsetWidth], [offsetHeight], [offsetPosX], [offsetPosY] )
```

At the end of the compiling process Photoshop generates a sprite image (named "sprites.png") and adjust width-, height-, background-image-, background-position- and background-repeat- properties. If a retina image is available in file system (@2x suffix) also a retina sprite image ("sprites@2x.png") will be generate.

Parameter	Description	Default Value
url	The image url relative to current CSS file	./.
offsetWidth	An integer value which is added to the calculated width.	0

offsetHeight	An integer value which is added to the calculated height.	0
offsetPosX	An integer value which is added to background-position-X.	0
offsetPosY	An integer value which is added to background-position-Y.	0

Typically only the url parameter is used. In special cases it is necessary to change width, height and background-position to adjust the image, e.g. if the top- or left-property is not zero.

6.5.7 Preloading Images

A CSS3 trick to define a class to preload images via a special property look's like this:

```
div.Preload{
  Background-image:url(anyimage1.png) -99999px -99999px;
  Background-image:url(anyimage2.png) -99999px -99999px;
  display:none;
}
```

μCSS includes a method to create a rule to preload images automatically:

6.5.7.1 μ.PreLoadImage

Define an image which has to preload by CSS file.

```
μ.PreLoadImage ( url )
```

Parameter	Description	Default Value
url	image url relative to current CSS file.	./.

6.5.8 Cursors

There two methods to define cursor properties. First the method μ.DefCursor is used in the μcss-init-rule to define a cursor and then the method μ.Cursor is used to create a concrete property in other blocks by the defined name, e.g.:

```
::-μcss-init{
  -μcss:μ.DefCursor("nooperation","default","imgs/cursors/nooperation.png",10,10);
  -μcss:μ.DefCursor("pointer","default","imgs/cursors/pointer.png",5,2);
}
a{
  -μcss:μ.Cursor("pointer");
  cursor:url(imgs/cursors/pointer.png) 7 2,pointer;
  cursor:image-set(url(imgs/cursors/pointer.png)1x,url(imgs/cursors/pointer@2x.png)2x) 7 2,pointer;
}
```

6.5.8.1 μ.DefCursor

Define a Cursor by a name and an image url. The offset parameter are used to define the center of the cursor image.

```
μ.DefCursor ( name, stdName, url, [offsetPosX], [offsetPosY] )
```

Parameter	Description	Default Value
name	The name of the cursor which is used to create concrete cursor properties with method μ.Cursor.	./.
stdName	Standard cursor name which is used on none conform browsers (W3C cursor names).	./.
url	URL of image.	./.
offsetPosX	Horizontal center position of image.	0
offsetPosY	Vertical center position of image.	0

6.5.8.2 μ.Cursor

This method creates a concrete cursor property. The cursor has to be defined via the method `μ.DefCursor`.

```
μ.Cursor ( name )
```

Parameter	Description	Default Value
name	Cursor name which is used in <code>μ.DefCursor</code> method.	./.

6.5.9 File System Operation Methods

Of course there are fine objects already in Photoshop Script which you can use to take file operations. But `μCSS` provides some comfortable methods including logging.

6.5.9.1 μ.OpenFile

6.5.9.2 μ.CopyFile

6.5.9.3 μ.DeleteFile

6.5.9.4 μ.CopyFolder

6.5.9.5 μ.DeleteFolder

6.5.10 FTP Methods

`μCSS` includes comfortable FTP methods.

Please note that it is possible to

To prevent direct coding of passwords, the JavaScript *prompt* function can be used:

```
-μcss:μ.$.pwd=Prompt(„Please enter FTP password for 192.168.0.1 and user ‚Test‘:“);  
-μcss:if(μ.$.pwd!="")SendFTPFile(„192.168.0.1“, 21, „Test“, μ.$.pwd, „mypic.jpg“, „mypic.jpg“);
```

6.5.10.1 μ.SyncFTPFolder

This method synchronize a directory on a FTP server with a local Folder.

```
μ.SyncFTPFolder ( host, port, user, pswd, ftpFolderPathName, localFolderPathName,  
[recursive], doFilePattern], [doNotFilePattern], [doFolderPattern], [doNotFolderPattern] )
```

If necessary local files will be sent to the FTP server or will be deleted if no corresponding local files exists. In recursive mode also all sub directories will be synchronized, where non local existing directories will be removed and local existing folders will be transferred to the FTP server.

Parameter	Description	Default Value
host	String with FTP host IP or name.	./.
port	Integer with the communication port of the connection.	./.
user	String with user name.	./.
pswd	String with the password for the connection.	./.
ftpFolderPathName	String with pathname of the FTP directory which should be synchronized with the local folder.	./.

localFolderPathName	String with pathname of the local folder which content should be transfered to the FTP directory.	./.
recursive	Optional, a boolean, true, if sub folders should be synchronized.	true
doFilePattern	Optional, a regular expression to define the file name patterns which will be added to the file names to have to synchronize.	./.*
doNotFilePattern	Optional, a regular expression to define the file name patterns which will NOT be added to the file names to have to synchronize.	/^\$/
doFolderPattern	Optional, a regular expression to define the folder name patterns which will be added to the folder names to have to synchronize. (Not used if <i>recursive</i> parameter is set to <i>false</i>).	./.*
notDoFolderPattern	Optional regular rxpression to define the folder name patterns which will be added to the folder names to have to synchronize. (Not used if <i>recursive</i> parameter is set to <i>false</i>).	/^\$/

6.5.10.2 μ .SyncLocalFolder

This method synchronizes a local folder with a directory on a FTP server.

```
 $\mu$ .SyncLocalFolder ( host, port, user, pswd, localFolderPathName, ftpFolderPathName,  
[recursive], [doFilePattern], [doNotFilePattern], [doFolderPattern],  
[doNotFolderPattern] )
```

Parameter	Description	Default Value
host	String with FTP host IP or name.	./.
port	Integer with the communication port of the connection.	./.
user	String with user name.	./.
pswd	String with the password for the connection.	./.
localFolderPathName	String with pathname of the local folder which should be synchronize with the FTP directory.	./.
ftpFolderPathName	String with pathname of the FTP directory which content should be transfered to the local folder.	./.
recursive	Optional, a boolean, true, if sub folders should be synchronized.	true
doFilePattern	Optional, a regular expression to define the file name patterns which will be added to the file names to have to synchronize.	./.*
doNotFilePattern	Optional, a regular expression to define the file name patterns which will NOT be added to the file names to have to synchronize.	/^\$/
doFolderPattern	Optional, a regular expression to define the folder name patterns which will be added to the folder names to have to synchronize. (Not used if <i>recursive</i> parameter is set to <i>false</i>).	./.*
doNotFolderPattern	Optional regular rxpression to define the folder name patterns which will be added to the folder names to have to synchronize. (Not used if <i>recursive</i> parameter is set to <i>false</i>).	/^\$/

7 μμ Object

For every CSS file a new μ object is created. If there are multiple CSS files selected for compiling, the $\mu\mu$ object is used to store information for all CSS files. This global object is valid while the μ CSS script is running.

7.1 $\mu\mu.\$$ Object

This object is free to use for any user properties. It is usefully to remember information over all compilations, e.g. when CSS files for multiple layouts should be created with general parameters for all layouts.

7.2 $\mu\mu.topCSSFiles$

This object property contains all CSS files which are selected by the user to compile.

8 Working with Colors

8.1 Defining Colors

The internal representation of a color in μCSS is an integer of form

- Bits 25 to 32: alpha channel value (0= transparent, 255=opaque)
- Bits 17 to 24: red channel value (0==dark, 255=bright)
- Bits 9 to 16: green channel value (0==dark, 255=bright)
- Bits 1 to 8: blue channel value (0=dark, 255=bright)

In μCSS-methods it is possible to set color parameters in different ways:

- As 32-bit integer value (like the internal definition above, e.g. 0xffff0000 for opaque red)
- A string in normal “#”-notation with two hex digits for each base color (e.g. “#00ff00” for full green), alpha channel value is automatically set to 0xff (opaque).
- A string in short “#”-notation with one hex digit for each base color (e.g. “#0f0” for full green), alpha channel value is automatically set to 0xff (opaque).
- A string in extended “#”-notation with two hex digits for each base color and alpha channel value (e.g. “#800000ff” for translucent blue).
- A string in decimal based absolute and/or relative CSS-rgb color notation (e.g. “rgb(255,0,0)” for opaque red). Absolute values in range of 0 up to 255, relative values in range from “0%” to “100%”. Values out of range will be clipped. Alpha channel value is automatically set to 0xff (opaque).
- A string in decimal based color absolute and/or relative CSS-rgba color notation with a float alpha definition (e.g. “rgba(255,0,0,0.5)” for translucent red). Absolute color values in range of 0 up to 255, relative color values in range from “0%” to “100%”. Alpha channel values in range of 0.0 to 1.0 or “0%” to “100%”. Values out of range will be clipped.
- A string in CSS-hsl color notation (e.g. “hsl(120,100%,25%)” for opaque dark green). This values are valid: hue color values in range of 0 up to 360, saturation values in range from “0%” to “100%” and lightness values in range from “0%” up to “100”. Values out of range will be clipped, hue values are turned to range 0 to 360.
- A string in CSS-hsl color notation with alpha channel value (e.g. “hsl(120,100%,25%,0.5)” for translucent dark green). This values are valid: hue color values in range of 0 up to 360, saturation values in range from “0%” to “100%”, lightness values in range from “0%” up to “100%” and alpha channel values in range of 0.0 to 1.0 or “0%” to “100%”. Values out of range will be clipped, hue values are turned to range 0 to 360.
- String color constants are supported (W3C CSS Color Module Level 3, extended color keywords), e.g. “red”, “Red” or “RED”.
- μ-color constants are supported (based on W3C CSS Color Module Level 3, basic color keywords with additional constants of μ.TRANSPARENT, μ.TRANSLUCENT and μ.OPAQUE), e.g. μ.RED, μ.LIME.

```
-μcss:μ.SetColor(0xff0000); /* set color to red */
-μcss:μ.SetColor("#f00); /* set color to red */
-μcss:μ.SetColor(0x80ff0000); /* set color to translucent red */
-μcss:μ.SetColor("#00ff00"); /* set color to green */
-μcss:μ.SetColor("#ff00ff00"); /* set color to opaque green */
-μcss:μ.SetColor("rgba(0,255,0,0.5)"); /* set color to translucent green */
-μcss:μ.SetColor("rgba(128,128,50%,0.5)"); /* set color to translucent 50% gray */
-μcss:μ.SetColor(μ.RED); /* set color to red */
```

```
-μcss:μ.SetColor("red"); /* set color to red */  
-μcss:μ.SetColor("Red"); /* set color to red */
```

8.2 Working with Alpha Channels

There are four ways to define an alpha channel value:

- As a float value between 0.0 and 1.0 (0.0=transparent, 1.0=opaque), values out of range will be clipped.
- As integer value between 0x00 and 0xff rep. 0 and 255 (0x00=transparent, 0xff=opaque).
- As hex string #00 and #ff ("#00"= transparent, "#ff"= opaque), values out of range will be clipped.
- As a percent string in range of "0%" up to "100%" ("0%"=transparent, "100%"= opaque), values out of range will be clipped (except in W3C *rgba* notation).
- As text "transparent" (=0%) or "opaque" (=100%) or "translucent" (=50%).

8.3 Color Calculations

There are many methods in μ object to make color calculations.

9 Plugins and Extensions

μCSS can be extended by plugins and extension. Plugins are general extensions for μCSS and will be placed in the plugin folder of μCSS, where extensions are individual parts which will be linked to a given CSS file and includes special functions. Extensions are placed normally at the same place where the CSS files are placed.

9.1 ButtonCreator PlugIn

This plugin realized an easy way to create icon buttons for different layouts where all layout variations are saved into one Photoshop file. There is only one method:

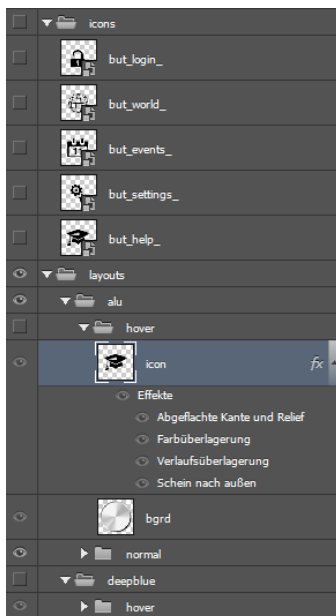
9.1.1 Create

This method creates all button images of a layout.

```
μ.plugins.ButtonCreator.Create ( [psdPathName], [layout], [savePath], [retinaMode] )
```

In RetinaMode also images in double resolution for retina displays will be generated.

Parameter	Description	Default Value
psdPathName	The relative pathname to the Photoshop image.	./.
layout	The Name of the layout corresponding to the layer group in Photoshop image.	./.
savePath	Path where the created images should be saved.	CSS output path
retinaMode	If true, two versions are created, one retina version (@2x) and a normal version. The normal version is resized by 50% in RetinaMode.	false



The Photoshop image has to be a special structure with named layers and group layers. The layer group with the name *icons* on top level contains all icons for the buttons. The name of each icon is the prefix for the file name of a created button.

The group layer with the name *layouts* on top level contains group layers for each layout. The group layers inside this group layer have to be named by the corresponding layout name. This layout name is the same which is used as parameter *layout* in method *Create*. Inside a layout group layer the button templates are defined. The name of the group layer is the suffix of the file name of a created button. One of the layers inside has to be named *icon*. The blending options of this layer will be copy to the layers in group layer *icons* when the images are created.

Inside a button state group layer all layer which should be visible have to set to visible. The plugin does not change the visibility of these layers.

All images will be saved as png-format. **Before the script starts you have to save your work because the script will close the document without saving.** If the document is not open in Photoshop the script will load the document automatically.

If the output path does not exist, the missing folders will be created.

Example of use:

```
::-μcss-init{  
  -μcss:μ$.layout="alu";  
  -μcss:μ.plugins.ButtonCreator.Create("buttons.psd",μ$.layout,"imgs/"+μ$.layout,true);  
}
```

10 Future Expansions

There are many ideas to improve μCSS in the future:

- More plugins.
- Optimization of compiled CSS files, e.g. collect single property definitions to shorter versions like border-style, border-color and border-width to a single border definition.

11 Release History

Date	Version	Release Notes
2013-03-25	0.9.0	First version.
2013-04-07	0.9.?	...development proceed...

12 Legal Information

These scripts are published under [MIT](#) license and are for free and commercial use. Neither Dongleware nor the author can't be held liable for any damages by using these scripts may cause.

The use of this software is on your own risk. Save your works before using these scripts.

Written by Meinolf Amekudzi.

©2005-2013



Dongleware Verlags GmbH
Am Güterbahnhof 18
69181 Leimen
Germany
FON +49 6224 17 52 40

USt-ID: DE 152 563 028
Steuernummer: FAHD 32492/11803
Handelsregister: Amtsgericht Mannheim HRB 334574
Sitz der Gesellschaft: Leimen
Geschäftsführer: Meinolf Amekudzi
www.dongleware.com

12.1 MIT Licence Terms

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

12.2 Trademarks

Photoshop is a registered trademark of Adobe Inc. Dongle is a registered trademark of Meinolf Amekudzi. The use of names and products is only for informational purposes and does not constitute a misuse of such trade names or trademarks. Dongleware Verlags GmbH assumes no guarantee in connection with the choice or use of these symbols.

12.3 Included Libraries

μCSS includes some libraries and/or algorithms from libraries written by third parties.

12.3.1 json.jsxinc

Based on json.js, ©2010 Douglas Crockford.

This code was downloaded from <https://github.com/douglascrockford/JSON-js>.

12.3.2 Bitpack.jsxinc

Based on node-bin-packing, ©2011 Jake Gordon and contributors.

This code was downloaded from <https://github.com/jsmarkus/node-bin-packing>.

13 Donate

We hope that this software is useful for you. If you'd like to support future development, you are welcome to make a donation via PayPal. Please use the following button to make a donation from your PayPal account. Any amount is welcome.



Thank you!